



# Too Much Sharing, Too Little Security: Authentication Cookie Theft At Scale

Tobias Gattermayer  
Technical University of Darmstadt  
Darmstadt, Germany  
ATHENE  
Darmstadt, Germany  
tobias.gattermayer@tu-darmstadt.de

Haya Schulmann  
Goethe University Frankfurt  
Frankfurt, Germany  
ATHENE  
Darmstadt, Germany  
schulmann@em.uni-frankfurt.de

## Abstract

Web browsers store cookies to save a user's authentication status. Sharing these authentication cookies across subdomains, a common practice for user convenience, creates a critical session hijacking vulnerability. When cookies for a parent domain are sent to all subdomains, an attacker who compromises any single subdomain can intercept them. Using these stolen cookies, the attacker can impersonate users across all other services hosted on the domain.

We conduct the first ethical, large-scale evaluation of authentication cookie leakage to compromised subdomains caused by insecure cookie scoping, a vulnerability we call "CookieDrift". We develop a novel, non-intrusive, and ethical method to detect CookieDrift by simulating a hijacked subdomain and recording all leaked cookies. From a list of over 2,500 popular domains with accounts, 57.98% (1,472) are vulnerable to cookie theft. Of these, 45.45% (669 domains) are immediately exploitable through subdomain takeover attacks. We find evidence of active, real-world hijacks affecting 186 domains and discover authentication cookies from these domains in darknet leaks using "CookieDelta", a new filtering technique we developed to isolate authentication-relevant cookie names. We investigate root causes of insecure cookie sharing by reviewing RFCs and major authentication frameworks and conclude: Defaults are secure, but examples encourage excessive cookie sharing caused by limitations in RFC 6265. We propose countermeasures that preserve backward-compatibility and ease-of-use while enabling finer-grained control over cookie propagation. We include two key artifacts: a locally running web application, "DeltaInterceptor", for testing any website's vulnerability to CookieDrift as well as computing CookieDelta, and a sanitized dataset of all recorded cookies.

## CCS Concepts

• **Security and privacy** → **Browser security; Web protocol security; Web application security.**

## Keywords

Web Application Security, Cookie Sharing, Dangling DNS, Subdomain Enumeration, Subdomain Takeover, Session Hijacking

## ACM Reference Format:

Tobias Gattermayer and Haya Schulmann. 2026. Too Much Sharing, Too Little Security: Authentication Cookie Theft At Scale. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792409>

## Resource Availability:

The source data and code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18277281>.

## 1 Introduction

Websites use HTTP cookies to save login states, reduce traffic, and improve performance. These cookies may be shared across subdomains to maintain user sessions. Subdomains (subdomain.org.com) of an organization's second-level domain (org.com), are generally considered trusted components and often reflect organizational hierarchy. This rule applies to most domains, except for services that delegate subdomains to users<sup>1</sup> or service endpoints<sup>2</sup>. Deliberate cookie sharing across subdomains expresses an implicit trust in the subdomains within the same organization. Critically, any subdomain that an adversary can directly or indirectly control becomes a potential avenue for session cookie hijacking, if authentication cookies from the parent domain are shared. It is practical for adversaries to take control over subdomains through several common attack vectors, including: ① exploiting vulnerabilities of the subdomain's server or in an integrated third-party service<sup>3</sup>; ② abusing infrastructure misconfigurations; ③ targeting internal services that have been decommissioned or reassigned; and ④ hijacking subdomains via *stale DNS pointers*.

Our goal is to explore the attack surface that results from current cookie sharing practices combined with adversary-controlled subdomains. While the threat is significant, its practicality is not yet well understood due to two main challenges. ① **Ethics**: A practical attack against a target domain involves control of a live subdomain and the theft of real users' cookies. Accessing private user data or deceiving users into visiting compromised sites is impermissible in a scientific context. Indeed, due to the ethical and technical challenges involved, these questions have not been answered prior to our work. ② **Scale**: Exploiting subdomains at scale is impractical, since locating and compromising individual vulnerable subdomains across a large set of domains is prohibitively time-consuming.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792409>

<sup>1</sup>[\[user\].wordpress.com](#), [\[username\].blog.fc2.com](#)

<sup>2</sup>[\[bucket-name\].s3.amazonaws.com](#)

<sup>3</sup>WordPress at [blog.org.com](#) or a Shopify at [store.org.com](#)

We use the term “CookieDrift” to describe an attack where an adversary intercepts cookies that have been shared with a compromised subdomain. Since it is unethical to find and exploit misconfigurations or vulnerable services running on arbitrary subdomains to access cookies, we focus our evaluation of this attack on simulating stale DNS pointers, also called dangling DNS pointers. These DNS records point to a resource, such as a cloud service or IP address, that has been de-provisioned or removed. Dangling subdomains as the attack vector allow us to develop an ethical method to evaluate cookie theft risks in the wild, without having to interact with real users or potentially cause service disruptions. We test 2,539 websites with user accounts for authentication cookie theft, selected from the top 10,000 of the latest list of one million popular domains. We find that 1,472 domains leak cookies, out of which 420 leak cookies relevant for authentication. These results reveal that standard cookie sharing practices strike the wrong balance between cookie security and convenience for web developers, because current specifications make it difficult to share cookies to subdomains in a controlled and granular way.

Authentication cookie theft exposes organizations to intrusion. By replicating stolen cookies in their browser, an adversary can impersonate a victim and gain access to a target organization or service. This method is called “session hijacking”<sup>4</sup> and bypasses the need for credentials and Multi-Factor Authentication (MFA), because these cookies store a post-authentication state. It is highly effective for gaining covert access to organizational networks. Okta, an identity management provider, showed that between 2020 and 2022 session hijacking attacks more than doubled every year<sup>5</sup>.

#### Novelty, Contributions and Artifacts

- *Analysis of Cookie Sharing* shows that current RFC specifications lack the granularity needed to share cookies securely in §3.
- *CookieDrift* is a novel attack that exploits authentication cookie theft on hijacked subdomains (§5).
- *First ethical and large-scale evaluation* of authentication cookie leakage to subdomains of 2,539 popular domains (§5.2).
- *CookieDelta* is a novel method to isolate authentication-relevant cookie names efficiently and ethically (§5.3).
- *First evidence* of actively hijacked subdomains within the set of cookie-leaking domains (§6.1).
- *Darknet Cookie Leaks* reveal a significant correlation with domains we identified as vulnerable to the CookieDrift attack (§6.2).
- *Root-cause analysis*. Insecure cookie scoping is a developer’s choice, not a framework default (§7).
- *DeltaInterceptor* simulates CookieDrift for *any* domain and isolates authentication cookies using CookieDelta.
- *Artifacts* include source code and demo video for DeltaInterceptor<sup>6</sup> and the sanitized dataset of all cookies from 2,539 domains<sup>7</sup>.

**Organization.** The remainder of this paper is structured as follows: We review related work in §2 and provide background on the two core concepts: cookie sharing (§3) and dangling subdomains (§4). Next, in §5, we introduce and evaluate the CookieDrift attack. We analyze results and cross reference them with leaked cookie data from the darknet in §6. We then investigate, why cookie sharing

across subdomains is so widespread (§7). Finally, we recommend mitigations, including the use of our DeltaInterceptor tool (§8), and conclude in §9, followed by ethical considerations in §10.

## 2 Related Work

**Cookie Sharing.** Several studies explore session cookies and their impact on browser security [4, 15]. Authors use HTTP GET requests to collect cookies from publicly accessible Internet sources (surface web / clear web) and analyze the “Set-Cookie” header and cookie attributes (Secure, HttpOnly, and SameSite). In contrast, we collect and study authentication cookies using scripted browser automation<sup>8</sup> to access content behind logins (deep web). Authentication cookies allow logging in as the corresponding user, with the privileges and access assigned to that user. While some cookies for tracking and site configuration are available on the open web, authentication cookies require a user account, and are created only *after* a successful login, making their collection more challenging.

Newton was developed by Mundada et al. [12] to identify authentication cookies. Newton downloads logged-in URLs multiple times, each time removing a unique set of cookies from the request, and checks whether the removal affects the rendered page’s login state. There are two limitations to this approach. ① The number of server requests grows exponentially with the number of cookies. We find that 95% of domains set between 14 and 20 cookies and some more than 100, as shown in Figure 1. The number of tests required by Newton increases exponentially with the number of cookies set by a website. Testing a website which sets 20 cookies, assuming a latency of 100ms, would require months, and generate an excessive number of server requests. As a consequence, this method fails for websites that set a large number of cookies, such as Bank of America. ② Newton expects to find the username displayed on the logged-in page, so it will also fail on websites which don’t. Removing cookies from the set of sent cookies can also introduce latency when authentication cookies are missing or when the server waits for user input, further slowing down the process.

Our methodology for the identification of authentication cookies, “CookieDelta”, does not face these limitations, it is efficient, applicable to all websites and ethical, requiring only a single load of a logged-in and logged-out URL. CookieDelta results are more accurate with no false negatives, i.e. no authentication-relevant cookie is missed. However, our approach *can* produce a superset of cookies, potentially including ones not strictly necessary for authentication. Drakonakis et al. [7] develop a framework to check for authentication vulnerabilities and find that 10,000 websites send cookies over non-encrypted connections and many do not use security mechanisms, such as HTTP Strict Transport Security (HSTS). The authors note that Newton [12] can be used to isolate authentication cookies. Finally, since [7] implement automated account creation, they exclude most websites that require CAPTCHA solving to create accounts. Most popular websites employ CAPTCHAs. This work has a number of differences from our research. Most importantly, CookieDrift assumes that websites implement best practices and configure the cookies with the required flags (Secure, HttpOnly, Domain).

<sup>4</sup>[https://owasp.org/www-community/attacks/Session\\_hijacking\\_attack](https://owasp.org/www-community/attacks/Session_hijacking_attack)

<sup>5</sup><https://auth0.com/resources/whitepapers/2022-state-of-secure-identity-report>

<sup>6</sup><https://zenodo.org/records/18277281>

<sup>7</sup><https://zenodo.org/records/18277281> - .zip Password: autho123

<sup>8</sup><https://pptr.dev>

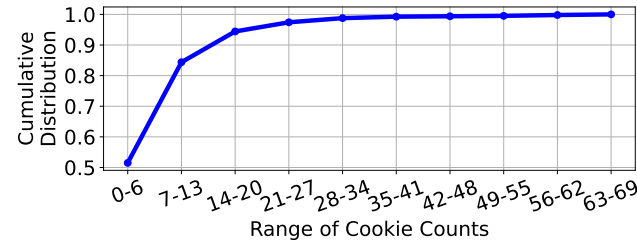


Figure 1: Total cookie count per domain when logged in.

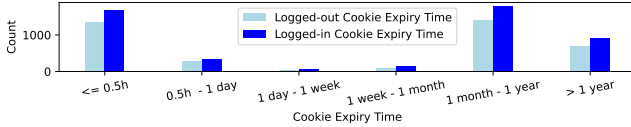


Figure 2: Leaked client-side cookie expiry.

Squarcina et al. [16] highlights the risk of cookie theft on subdomains, but does not provide a quantitative evaluation. Furthermore, our focus is on authentication and we choose interactive account creation to solve CAPTCHAs manually but consistently, allowing us to collect a dataset of real-world authentication cookies, which is not feasible with the fully automated methods used by [16]. We then introduce a novel approach to filter authentication-relevant cookies and conclude that the widespread unintentional sharing of cookies stems from RFC specifications, which make it difficult for developers to selectively restrict cookie access to subdomains.

**Dangling DNS Pointers.** Previous research measured different resources, e.g., IP addresses and cloud buckets, pointed to by dangling records [2, 9, 11, 14, 16, 18] and showed that acquiring control over released resources is practical. Applying this concept, we simulate control of a random dangling DNS record in a local environment using accounts we create to determine whether, and to what extent, a domain leaks cookies to subdomains.

### 3 Cookie Sharing

Cookies are key-value pairs that a web server sets, and that web browsers include in the Cookie header of subsequent HTTP(S) requests. They are also transmitted via the Set-Cookie header from the server to the client. This mechanism establishes and maintains a synchronized session state in the otherwise stateless HTTP protocol defined in RFC 6265<sup>9</sup>. Cookies are used to track users across pages via a session identifier and to store authentication tokens that signal to the server that a user is logged in. Consequently, stealing authentication cookies and using them in another browser can grant full access to the site as that user. Cookie attributes were introduced to secure and protect sensitive data across sessions. Three cookie attributes are relevant for leakage and exfiltration using hijacked subdomains: HttpOnly, Secure and Domain.

**Client-side cookie attributes** reveal the expiry times of the cookies shown in Figure 2. While many are short-lived (less than

one hour), a significant portion persists for over a month, providing attackers with a long window for exploitation. We assume the implementations of cookie-based authentication comply with contemporary best practices<sup>10</sup> with the HttpOnly and Secure attributes set, 18.28% (1,200 out of 6,563) of logged-in cookies include HttpOnly. Going forward, we focus on the Domain attribute, which facilitates intra-domain sharing of cookies.

**Cookie Sharing via the Domain Attribute.** By default, cookies are only accessible on the originating hostname. The Domain attribute allows sharing them across subdomains (Figure 3).

**Supported cookie sharing** (current RFC 6265 / past RFC 2109)

- ① Cookie is created and accessible on current subdomain sub.org.com only, without the Domain attribute set.
- ② Cookie is created, but not accessible on current subdomain sub.org.com and shared with all its subdomains sub.sub.org.com. *Supported in RFC 2109, no longer supported in RFC 6265.*
- ③ Cookie is shared with subdomain sub.org.com and all its subdomains sub.sub.org.com.

**Unsupported cookie sharing** (current RFC 6265):

- ⑤ Cookie is shared with subdomain sub.org.com, but not its subdomains sub.sub.org.com.
- ⑥ Shared with subdomain and some of its subdomains.
- ⑦ Cookie is created on subdomain, shared with *some* subdomains.
- ⑧ Cookie is shared with entire level of subdomains, not superdomain. *This behavior was supported in RFC 2109.*
- ⑨ Cookie is shared with some subdomains, not superdomain.

**The leading dot’s** meaning as a prefix to a single subdomain name as a value of the Domain attribute has disappeared over time. Prior to RFC 6265, the presence of the leading dot meant that a cookie is made explicitly available to subdomains. However, after RFC 6265 (§4.1.2.3, §5.2.3) became the standard in 2011, modern browsers, as well as other user-agents like cURL<sup>11</sup>, started treating a cookie’s domain attribute that is set for blog.org.com and .blog.org.com as equivalent, ignoring the leading dot and *always* sharing the cookie with the domain specified in the attribute and all of its subdomains. The decision to ignore the meaning of the leading dot was made due to its inconsistent browser interpretation at the time and was part of an overall effort to bring “simplified semantics<sup>12</sup>” to cookie attributes. Directly attributable to the guidelines established by RFC 6265, the “sharing by default” interpretation of

<sup>10</sup><https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies#>

<sup>11</sup><https://github.com/curl/curl/blob/master/lib/cookie.c>

<sup>12</sup><https://datatracker.ietf.org/doc/html/rfc6265#section-4.1.2>

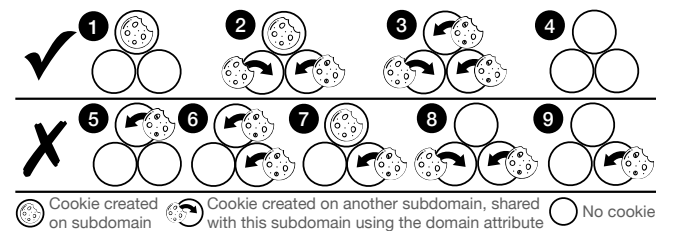


Figure 3: Cookie sharing (RFC 6265).

<sup>9</sup><https://datatracker.ietf.org/doc/html/rfc6265>

the Domain attribute results in less granular control for developers and causes wider cookie distribution, which is exploitable on hijacked subdomains.

In summary, the Domain cookie attribute lacks the granularity to properly balance security and convenience, resulting in excessive and often unwanted cookie sharing across subdomains and making the CookieDrift attack a widespread threat.

## 4 Dangling DNS Pointers

A "dangling" or "stale" Domain Name System (DNS) record stores a pointer to a resource that is no longer active. An attacker can re-register the target resource with control over its content while the stale DNS pointer is reactivated enabling cookie theft [1, 2, 10, 11, 16, 18]. Research shows that dangling DNS records are common, most recently Zhang et al. [18] discover 10K dangling subdomains. Next, we show how different types of DNS records can be reactivated.

**Reactivating DNS NS records.** There are two ways for an attacker to reactivate dangling name server (NS) records. ① By claiming a deprovisioned zone on a shared provider platform (like AWS Route 53) where a stale delegation exists, ② By re-registering the expired domain of a name server itself. Regardless of the method (direct account compromise or the exploitation of abandoned records), an attacker can install web servers on compromised subdomains that intercept shared authentication cookies from the parent domain when logged-in users visit the site.

**Reactivating DNS A records.** A subdomain-takeover vulnerability can also arise when a DNS A record resolves to an unallocated IP address within a cloud provider's dynamic allocation range. Liu et al. [11] show, that an adversary can provision a new virtual machine to acquire a specific IP address in an IP lottery system, thereby hijacking the subdomain with full root access to the server instance.

**Reactivating DNS CNAME records.** A dangling CNAME pointer occurs when a subdomain points to a third-party service that is later deprovisioned, but the DNS pointer (blog.org.com  $\mapsto$  orgblog.wordpress.com) persists. An adversary re-registers the same service name (orgblog.wordpress.com), gaining administrative control of the new service instance. Although this method does not result in root-level web server access, application-level administrator privileges on a content management system like WordPress are sufficient to execute server-side code, injected into themes or plugins, to exfiltrate cookies from site visitors (example in Appendix / Listing 2).

**Access level.** Re-registering an application-layer service such as an Amazon S3 bucket that provides web hosting via a DNS CNAME (blog.org.com  $\mapsto$  [freetext].s3-website-us-east-1.amazonaws.com) provides only static content hosting capabilities with insufficient server-side access to exfiltrate HttpOnly cookies. Therefore, the ability to access authentication cookies, which have the HttpOnly flag set, depends on the level of access an attacker has on the re-activated service (Appendix / Table 4).

## 5 The CookieDrift Attack

The CookieDrift attack occurs when an attacker hijacks any subdomain to access authentication cookies that are shared across all subdomains. For ethical evaluation against the top 10,000 domains

of the Alexa domain list, we create 2,539 user accounts. We then access and save real-world authentication cookie data, if present, from these user accounts before and after the login process.

**Authentication Cookies** store a unique identifier, enabling recognition of a user session across multiple related requests by the server. Upon login, the server sends this ID as the value of a cookie; the browser then returns it with each request, maintaining a logged-in state. Logging out, session expiration, or server-side invalidation transition the session state to logged-out, requiring re-authentication. Safeguarding these cookies is vital to prevent session hijacking, which is what we demonstrate with the CookieDrift attack. We develop a new method we call CookieDelta to identify only authentication-relevant cookies in §5.3.

**Attack Methodology.** CookieDrift exploits cookie sharing to steal authentication cookies on a subdomain. The Secure and HttpOnly attributes enforce encrypted data transfer and restrict browser access to cookies. For this reason an adversary requires server control, more specifically the ability to inject server-side code, which sends HttpOnly and Secure cookies back to them (Appendix / Listing 2 in the Appendix is an example in PHP). This code executes every time a page is rendered in response to a browser request. The response includes the user's authentication cookies which are accessible on the attacker-controlled server, enabled by the Domain attribute. In summary, executing exfiltration code for server-side cookies requires a web server's backend context on a subdomain with which cookies are shared.

**Ethical Simulation of CookieDrift.** We replicate this setup using a local, HTTPS-enabled web server as described in §5.2. The steps for exploiting a hijacked subdomain to steal authentication cookies are outlined below and illustrated in Figure 5 (A).

- ① Adversary sets up web server on the hijacked subdomain, which hosts a phishing website with the goal of stealing the authentication cookies of logged-in users.
- ② User logs in via any of the domain's authentication mechanisms, after which authentication-relevant cookies are stored in their browser's local cookie storage.
- ③ User receives a phishing message via email or other means, incentivizing them to click on the *legitimate-looking* link to the adversary's phishing site that was setup in step ①.
- ④ User clicks the link and visits the malicious site.
- ⑤ Malicious site loads in the user's browser and all cookies shared with the subdomain are exfiltrated and saved by the adversary.
- ⑥ Adversary adds authentication-relevant cookies to their web browser, impersonating the original user and inheriting the same level of access to organizational resources, thereby gaining a strong initial foothold in the network.

### 5.1 User Account Creation

We choose manual account creation for two reasons: ① Many websites implement measures to deter automated account creation as a means to combat spam and we aim to develop an ethical study that does not disrupt the service in any way. ② To our knowledge, there are no universally effective automated methods to reliably solve modern CAPTCHAs implemented by these websites. Indeed, we find that most websites with accounts implement CAPTCHAs.

Consequently, previous efforts to automate account creation excluded websites with CAPTCHAs from their study [7]. We reuse the same credentials and profile information to systematically register accounts on target services and complete email validation to ensure all accounts are fully instantiated. Similar to the methodology in [7], we use SSO-based identity providers (e.g., “Login with Google” or “Login with Facebook”) when available to streamline the registration process. This approach yields an average rate of 28 new user accounts per hour. We audit 10,000 Alexa domains and create 2,539 accounts in approximately 72 person-hours. Figure 4 shows the popularity rank distribution of these domains with accounts.

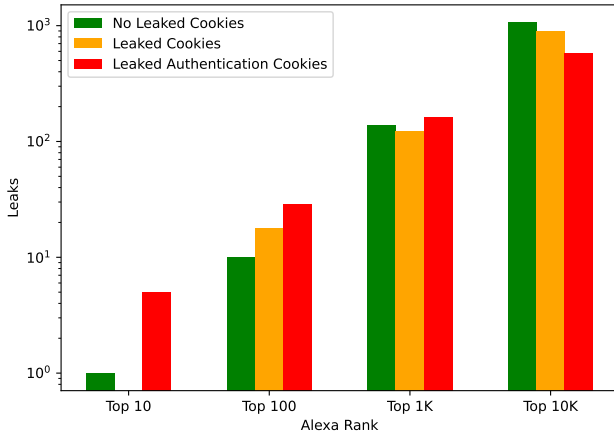


Figure 4: Cookie leaking website ranking.

## 5.2 Evaluation

The unethical nature of running tests on live websites necessitates an ethical alternative to evaluate the CookieDrift attack via a simulated subdomain hijack (Figure 5 (B)). We redirect a random subdomain of a real domain (`hijacked.org.com`  $\mapsto$  `127.0.0.1`) and host it locally. Cookie leakage to this fictitious subdomain implies leakage to *all* Internet-facing subdomains. The following steps outline the test setup:

- 0 **Simulation Setup.** We run a web server with a generic SSL certificate on `127.0.0.1` (localhost) and point every test subdomain (called `hijacked.sld.tld`) to the server and override the system’s DNS resolution by appending an entry for each subdomain to the `/etc/hosts` file.  
Adding `--ignore-certificate-errors` to launch Chromium-based web browsers avoids strict certificate checking and eliminates the need to create a valid certificate for every (fake) subdomain. Cookie leakage to the randomly named subdomain, which exists only within our local environment, implies cookie leakage across all (real) subdomains.
- 1 **Collect cookies when logged in.** We log in using the regular login page of the domain. Then we use another browser tab to visit the hijacked subdomain hosted on `127.0.0.1` to record all cookies from our web server.
- 2 **Collect cookies when logged out.** We log out using the logout functionality of the website and visit the hijacked

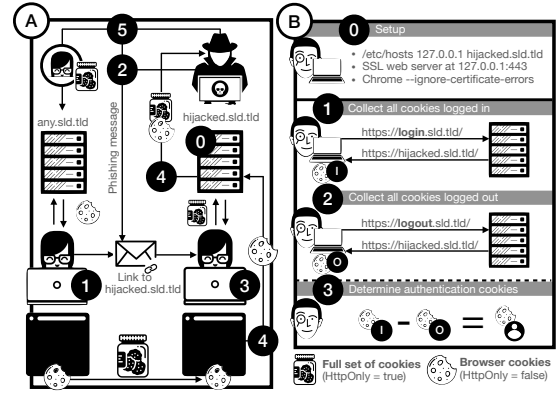


Figure 5: (A) CookieDrift (B) Ethical evaluation.

subdomain again, recording the set of cookies present in a logged-out state.

- 3 **Apply CookieDelta** to identify authentication-relevant cookies by subtracting the set of logged-in 1 cookies from the set of logged-out cookies 2.

Using this approach, we evaluate 2,539 cookie sharing configurations, yielding a dataset of cookies recorded while logged-in and logged-out. Next, we present a new method to isolate authentication-relevant cookie names.

## 5.3 CookieDelta

To characterize the real-world exposure of compromised user credentials, we must first identify the authentication cookie names for each domain. This allows them to be cross-referenced against cookie leak datasets sourced from darknet leaks (§6.2). However, determining the subset of authentication cookies from all cookies is challenging because neither their name nor their value reveals their function (e.g. authentication, tracking or site configuration). We assume authentication cookies are set using `HttpOnly`, and exclude browser-accessible tracking and site configuration cookies. Still, names of authentication cookies differ greatly between custom implementations and middleware (Appendix / Table 5). In §2 we explain that existing approaches for detecting authentication cookies are limited. Some try to identify them by searching for string patterns, e.g., ‘session’, ‘session\_id’, ‘PHPSESSID’, ‘JSESSIONID’, or ‘ASP.NET\_SessionId’, but doing so misses out on implementations that do not use such names [3]. The methods in [12] are time-consuming with HTTP requests that exponentially grow with cookie count. To improve these methods, we introduce “CookieDelta”, an ethical offline method that produces a set of authentication-relevant cookies directly correlated with the authentication event depicted in Figure 5 (B) / 3. CookieDelta produces all cookies that are created, modified, or deleted during the login process, thereby isolating session-critical cookie names across all website implementations. CookieDelta is guaranteed to include all authentication-relevant cookie identifiers and, at the very least, drastically reduces the initial set of cookies for slow online methods [12] that determine the definitive, minimal set. CookieDelta offers the first efficient, offline and ethical method to determine all

authentication-relevant cookies for arbitrary cookie-based authentication implementations.

**Accuracy.** We evaluate precision and recall by assessing the accuracy of CookieDelta using a subset of 100 randomly selected websites and extrapolating these results to the full dataset. For each website, we establish a *ground truth* by manually identifying which cookies are necessary for authentication. This is done by ① Logging in and out with test accounts; ② Observing which cookies change or are added during authentication; ③ Testing whether each cookie is required to re-establish a logged-in session after deletion. CookieDelta is then applied to this dataset, and its output compared to the ground truth to compute *True Positives (TP)*, *False Positives (FP)*, and *False Negatives (FN)*.<sup>13</sup> We observe the following counts over the 100 test websites: **TP**: 196; **FP**: 14; **FN**: 0. The 14 false positives are cookies associated with analytics or UI personalization that coincidentally changed during login.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = 0.933, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = 1$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \cdot \frac{0.933 \cdot 1}{0.933 + 1} \approx 0.97$$

CookieDelta achieves both high precision and recall. The result aligns with our experiments, which demonstrate similar effectiveness at identifying the full set of authentication-relevant cookies while minimizing noise. Applying these rates to the full dataset of our 420 authentication-cookie-leaking domains, assuming similar distribution, we expect 59 FPs:  $\approx 420 \cdot \frac{14}{100} = 59$ .

#### 5.4 CookieDrift Evaluation Results

We find 1,472 (almost 60%) of 2,539 domains with accounts that leak cookie data to subdomains (Appendix / Figure 7). CookieDelta identifies a subset of 420 domains leaking diverse authentication cookie counts between one and 26 depicted in Figure 6.

#### 5.5 Dangling Subdomain Enumeration

The CookieDrift attack is enabled by compromised subdomains that receive shared cookies. Therefore, our goal is to discover which of the 420 cookie-leaking domains also have subdomains pointed to by dangling DNS records.

**Subdomain Enumeration.** Our initial investigation reveals that different tools and methods produce non-overlapping sets of results

<sup>13</sup>TP (True Positive): CookieDelta correctly identifies an authentication cookie.  
FP (False Positive): CookieDelta identifies a non-authentication cookie as authentication-relevant.  
FN (False Negative): CookieDelta fails to identify a true authentication cookie.

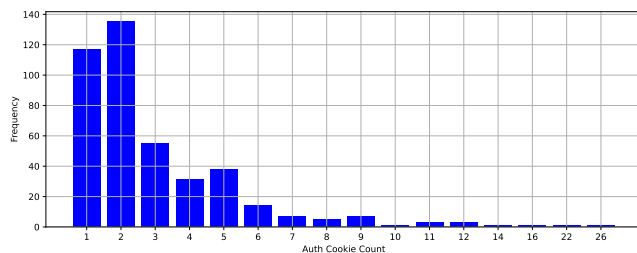


Figure 6: Auth cookie counts across 420 domains.

| Domain                      | SDBF    | GAN           | Subfinder     | Sublist3r | SubSpider      | New     |
|-----------------------------|---------|---------------|---------------|-----------|----------------|---------|
| airbnb.com                  | 73      | 52            | <b>1,629</b>  | 219       | 74             | 1       |
| digitaloceanspaces.com      | 2       | 3             | <b>2,129</b>  | 41        | 1,761          | 1,655   |
| dropbox.com                 | 24      | 244           | <b>2,027</b>  | 401       | 87             | 2       |
| facebook.com                | 723     | 696           | <b>12,046</b> | 286       | 294            | 214     |
| google.com                  | 150     | 134           | <b>5,640</b>  | 98        | 3,035          | 2,808   |
| herokuapp.com               | 30,624  | <b>30,699</b> | 739           | 4,501     | 9,988          | 9,952   |
| kindredgroup.com            | 5       | 6             | <b>139</b>    | 67        | 3              | 0       |
| logitech.com                | 51      | 46            | <b>363</b>    | 279       | 27             | 6       |
| mi.com                      | 5       | 210           | <b>1,365</b>  | 1,244     | 86             | 58      |
| nyc3.digitaloceanspaces.com | 915     | 854           | <b>990</b>    | 3         | 599            | 195     |
| sony.com                    | 45      | 46            | <b>7,887</b>  | 1,781     | 54             | 12      |
| ui.com                      | 20      | 75            | <b>2,559</b>  | 1,565     | 42             | 15      |
| wordpress.com               | 87,194  | 95,001        | 270           | 73        | <b>379,297</b> | 379,278 |
| yahoo.com                   | 425     | 561           | <b>10,455</b> | 1,483     | 1,497          | 1,174   |
| youtube.com                 | 43      | 60            | <b>740</b>    | 11        | 103            | 69      |
| zoom.us                     | 509     | 1,581         | <b>13,834</b> | 2,528     | 3,047          | 3,015   |
| <b>Total Subdomains</b>     | 120,808 | 130,268       | 62,812        | 14,580    | <b>399,994</b> | 398,454 |

Table 1: Subdomain enumeration comparison with the highest count for each domain in bold.

(Table 1). We therefore combine multiple techniques to maximize coverage of subdomain discovery to create the most complete set. The Subdomain Enumeration Guide<sup>14</sup> and [14] discuss several ways to accomplish this. We compare the subdomains found by two widely popular tools Subfinder<sup>15</sup> and Sublist3r<sup>16</sup> as well as two methodologies developed in previous research: SDBF (Smart DNS Brute-Forcer) [17] and a Generative Adversarial Network (GAN)-based method by [6]. [17] and the Subfinder and Sublist3r tools probe candidate subdomains via DNS brute-force against a target domain. Previous research [6] uses GANs to generate candidates for subdomain names, and checks them for validity. To compare performance, we use the list of 16 domains from [6].

The results of our evaluation are listed in Table 1. We find that, while Subfinder performs well for most domains, none of the existing techniques performs best for all. Interestingly, existing approaches all perform poorly on discovery of domains with a high number of web-accessible subdomains (e.g. wordpress.com), highlighting that leveraging URLs collected from crawling Internet content is a promising strategy to find different domains from Subfinder and Sublist3r (last column in Table 1).

To augment existing tools, we develop an ethical approach that utilizes up-to-date and web-accessible URLs that respect each site's robots.txt restrictions to filter subdomains independent of any other third-party service, utilizing openly available data from the Common Crawl Project<sup>17</sup>. We implement our own filtering program, dubbed "SubSpider", similar to the common-crawl-subdomains repository<sup>18</sup>.

Our approach is able to consistently identify new subdomains, which both Subfinder and Sublist3r, as well as the previous research [6, 17], missed and thus SubSpider presents a valuable addition to existing subdomain enumeration strategies. We conclude that combining methods provides the most comprehensive discovery of subdomains going forward. In Table 1 in the rightmost column "New", we list the new subdomains discovered by SubSpider, which were not discovered by any of the previous approaches for each of the domains on the list of 16 domains from [6]. We demonstrate that on all domains except one, SubSpider discovered subdomains that were not found by any of the other approaches.

<sup>14</sup><https://sidxparab.gitbook.io/subdomain-enumeration-guide/>

<sup>15</sup><https://github.com/projectdiscovery/subfinder>

<sup>16</sup><https://github.com/aboul3la/Sublist3r>

<sup>17</sup><https://www.commoncrawl.org/>

<sup>18</sup><https://github.com/carlbordum/common-crawl-subdomains>

| Domain            | Subdomains | Domain            | Subdomains |
|-------------------|------------|-------------------|------------|
| 1. wordpress.com  | 379,297    | 11. teachable.com | 12,924     |
| 2. weebly.com     | 69,609     | 12. bandcamp.com  | 12,799     |
| 3. medium.com     | 55,612     | 13. neocities.org | 11,136     |
| 4. fc2.com        | 45,494     | 14. gumroad.com   | 10,598     |
| 5. substack.com   | 23,821     | 15. overdrive.com | 10,346     |
| 6. eventbrite.com | 20,300     | 16. wix.com       | 8,758      |
| 7. jimdo.com      | 19,070     | 17. wikidot.com   | 7,685      |
| 8. deviantart.com | 18,233     | 18. etsy.com      | 7,058      |
| 9. podbean.com    | 16,698     | 19. gitbook.io    | 6,982      |
| 10. typeform.com  | 15,984     | 20. slack.com     | 5,663      |

Table 2: Top 20 subdomains of vulnerable domains.

Next, we apply the consolidated subdomain enumeration approach, i.e., all the methods in Table 1 including our own SubSpider, to discover the subdomains of the domains in our dataset.

**Discovering Domains with Dangling DNS.** We enumerate subdomains of each of the 2,539 domains with accounts using the consolidated subdomain enumeration methods in Table 1 and SubSpider to extract unique subdomains from the latest CDX index files<sup>19</sup> from the Common Crawl web data archive [5]. We collect 939,922 subdomains, Table 2 shows the highest subdomain counts.

Subsequently, we check which DNS records for these domains are dangling using the methodology developed in previous work [11]. Measuring the prevalence of the dangling records requires two steps: ① We send liveness probes to IP addresses, to which the domains are mapped, to determine if the IP addresses are allocated and in use. ② We check TCP ports 80 and 443 and classify records that point to IP addresses that do not respond as dangling (Appendix / Listing 1 for details).

We categorize 1.87% (17,565) of 939,922 subdomains as dangling. 75 domains out of 2,539 domains with accounts have a total of 744 dangling subdomains. Among the 1,472 domains with accounts that leak cookies, we find 669 dangling subdomains. Finally, 29 dangling subdomains receive leaked authentication-relevant cookies, as determined by CookieDelta. After reactivation, authentication information could be immediately exfiltrated from these subdomains following the steps of the CookieDrift attack in §5.

## 6 Real-World Exposure

In §6.1, we identify actively hijacked subdomains to which cookies are propagated and filter names of 3.6 billion cookies by authentication-relevant cookie names in §6.2 with CookieDelta.

### 6.1 Actively Hijacked Subdomains

We evaluate whether subdomain takeover attacks occur by comparing characteristics and metadata of HTML source code we download from the subdomains identified in the previous section. 31.2% of the 582 domains that leak authentication cookies had actively hijacked subdomains at some point during this period.

For every month in a period of three years, we download each subdomain’s sitemap, HTML code of the index page and DNS data extracted from URLs from Common Crawl<sup>20</sup>. We proceed by looking for changes in content over time, including the following characteristics: DNS records, WHOIS information, sitemap (e.g., drastic size differences), language, and extracted keywords from the HTML

| Auth-Cookie Leaking Domain | Hijacked Subdomain # | Auth-Cookie Leaking Domain | Hijacked Subdomain # |
|----------------------------|----------------------|----------------------------|----------------------|
| 1. microsoft.com           | 344                  | 11. mercedes-benz.com      | 17                   |
| 2. dynamics.com            | 63                   | 12. adobe.com              | 15                   |
| 3. hp.com                  | 51                   | 13. nvidia.com             | 15                   |
| 4. ey.com                  | 46                   | 14. ea.com                 | 13                   |
| 5. msn.com                 | 33                   | 15. cfainstitute.org       | 12                   |
| 6. autodesk.com            | 26                   | 16. cisco.com              | 12                   |
| 7. nsw.gov.au              | 25                   | 17. lenovo.com             | 12                   |
| 8. ford.com                | 22                   | 18. intuit.com             | 11                   |
| 9. abb.com                 | 19                   | 19. intel.com              | 9                    |
| 10. live.com               | 18                   | 20. walgreens.com          | 9                    |

Table 3: Top 20 SLDs with hijacked subdomains that leak authentication cookies in a 3 year timespan.

code (details of the algorithm in Appendix / Listing 1). We check the content and classify subdomains as hijacked only if these characteristics differ *significantly* from one month to the next. For example, we find gambling or adult content on corporate subdomains as an obvious mismatch. We consider our estimates to be conservative, because the detection method is parameter-sensitive, and we count only high-confidence cases.

Nevertheless, we detect 186 actively hijacked subdomains out of 582 domains that we identified as cookie leaking and 31.2% (131 out of 420) of domains had at least one actively hijacked subdomain at some point within three years (see details in Table 3). Domains owned by Microsoft (microsoft.com, dynamics.com, msn.com, live.com, office.com, office365.com, skype.com, xbox.com, visual-studio.com, microsoftonline.com) account for 46.9% (482 of 1,028) of hijacked domains.

### 6.2 Authentication Cookie Leaks

We investigate 3.6 billion cookies compiled from darknet sources between 2021 and 2023 to investigate. We previously identified 420 domains as vulnerable to CookieDrift out of a total set of 2,539. Using authentication cookie names filtered with CookieDelta, we can confirm that all 420 vulnerable domains have authentication cookies present in the leaks. These 420 vulnerable domains account for over 550M (550,041,733) leaked cookies, 24.36% more than the 442M (442,294,610) cookies found from the remaining 2,119 non-vulnerable domains. The disparity becomes starker when normalized. Vulnerable domains average over 1.3M leaked cookies (1,309,623), compared to about 208K (208,728) for non-vulnerable domains. Consequently, a cookie from a vulnerable domain is over six times more likely to appear in a leak. By category, “Search Engines” and “News & Media” (almost 20%) were most affected, while Brazil, Indonesia, and Vietnam led the country-based leaks, together accounting for nearly 20% of the total. Google domains constitute a significant portion, accounting for 93,127,780 leaked cookies. For the 420 vulnerable domains, we note that there were 9 times as many non-authentication cookies as authentication cookies in the dataset (see Appendix A for details).

## 7 Root Cause Analysis of Cookie Oversharing

To understand the prevalence of sharing authentication cookies across subdomains, we first examine the default session management settings of popular web frameworks and find that these frameworks<sup>21</sup> (e.g., ASP.NET, Express, Ruby on Rails) do not set the

<sup>19</sup>CC-MAIN-2023-40: <https://data.commoncrawl.org/cc-index/collections/index.html>

<sup>20</sup><https://data.commoncrawl.org/cc-index/collections/index.html>

<sup>21</sup><https://trends.builtwith.com/framework>

Domain attribute for cookies by default (see Table 5 in the Appendix). As a result, authentication cookies remain scoped to the specific origin (e.g., `auth.org.com`) rather than being accessible across all subdomains (e.g., `.org.com`). This default behavior aligns with secure design principles and suggests that the widespread cookie oversharing we observe is not caused by the frameworks themselves. Instead, it points to a deliberate developer configuration. To understand why developers might set the Domain attribute explicitly, we review official documentation and tutorials (Appendix / Table 6) for popular web stacks. Examples recommend using `Domain=.org.com` to share authentication states of users across all subdomains to enable seamless Single Sign-On (SSO).

## 8 Mitigations and Improvements

We demonstrate that while client-side solutions can help reduce the risk of the CookieDrift attack, only server-side strategies can fully mitigate it.

**CookieDelta in a browser extension** improves transparency of client-side cookie sharing and alert the user when authentication-relevant cookies (detected by CookieDelta) are shared with non-essential subdomains, prompting the user for explicit permission, for instance: “*christmas-blog.org.com is about to access your authentication data created by login.org.com. Do you want to allow this?*”.

**Token binding** binds a session cookie to a client, based on RFCs 8471, 8472 and 8473: during TLS the client proves possession of a non-exportable private key, and the server binds the cookie to that key. While off-device reuse of cookies is prevented, there is no protection for active sessions of a compromised server, thus, server-side mitigations remain decisive.

**Subdomain scoping** groups all services that require authentication cookies under a dedicated parent domain. By segregating services as `app1.secure.org.com` and `api.secure.org.com`, a session cookie can be narrowly scoped to `Domain=.secure.org.com`, preventing propagation to unrelated subdomains. While this containment can significantly reduce the attack surface for CookieDrift, it does not eliminate the risk entirely. A compromise of any server within the `.secure.org.com` scope would still allow an attacker to steal session cookies and use them to impersonate the associated user. Furthermore, retrofitting a new namespace can be infeasible for large legacy IT infrastructures.

**Proactive checking for dangling DNS entries** can prevent CookieDrift. IT administrators should ① Maintain an updated inventory of DNS records and perform regular security audits. ② Delete records pointing to discontinued services. ③ Identify existing hijacks by examining content of Internet-accessible services.

**DeltaInterceptor**<sup>22</sup> is our research tool for cookie security we provide (Appendix §B). It features a graphical interface and simulates the CookieDrift attack using browser automation and local DNS redirection to intercept and record both logged-in and logged-out cookie sets of any website. It performs and validates CookieDelta using live session hijacking and source code analysis, which utilizes a large language model to assess login success.

**Centralized DNS and hosting management** can provide robust protection against CookieDrift, an organization should centralize control over its DNS records and service hosting. A single

entity should manage the entire life cycle of these assets, ensuring DNS entries and their corresponding services are provisioned and de-provisioned in tandem. This central authority should then create a formal process for organizational units to request establishing and decommissioning hosting and DNS services.

A new “**Scope**” attribute can prevent cookie oversharing without breaking backward compatibility with an explicit, fine-grained allow-list of domains constraining the broader Domain attribute. Modern web browsers would enforce this whitelist, while legacy web browsers would ignore Scope, ensuring graceful degradation.

## 9 Conclusion

We introduce CookieDrift and perform the first large-scale, ethical study of cookie leakage caused by the insecure use of the Domain attribute. Prior to this work it was unclear how to investigate such threats ethically and how many domains are vulnerable. We develop a way to ethically evaluate the theft of authentication-cookies at scale against 2,539 domains that support user account creation and find that 1,472 domains are vulnerable to cookie theft, from 420 of which we are able to reproducibly steal *authentication cookies*. Additionally, 75 domains of these 1,472 cookie leaking domains have at least one subdomain pointed to by dangling DNS records. Finally, in 29 cookie-leaking domains we find evidence of subdomain hijack allowing live exploitation. darknet cookie leaks reveal that there are sixfold more leaked cookies from vulnerable domains compared to those that do not leak cookies, indicating that cookie-leaking domains are more commonly subject to cookie theft. Investigating the root cause, we find that sharing cookies to subdomains is *not* a default setting of authentication frameworks, but a choice guided by official developer documentations, which all propose setting `Domain=.org.com` to share a cookie to all subdomains. These findings advocate for revisiting cookie-sharing mechanisms and the possible addition of a Scope whitelist to the Domain attribute. Along with this work, we release the first version of DeltaInterceptor, a tool that enables investigation of cookie-sharing configurations for arbitrary domains (Appendix / Figure 13).

## 10 Ethical Considerations

This study was conducted under institutional IRB approval and followed established guidance for responsible scanning and data handling [8, 13]. Measurements avoid service disruption and prevent exposure of private user data. Subdomain enumeration relied on offline CommonCrawl data that honors `site robots.txt`. Tests used researcher-controlled random accounts only, and where available we used federated login to minimize site impact. A partner supplied a dataset of leaked cookies with values removed; we did not retain raw cookies and store only aggregate statistics. Processing aligns with GDPR Art. 6(1)<sup>23</sup>; data collection was minimized, temporary working data was protected and deleted after analysis.

## Acknowledgments

This work has been co-funded by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE.

<sup>22</sup><https://doi.org/10.5281/zenodo.18277281>

<sup>23</sup><https://gdpr.eu/>

## References

- [1] Eihai Alowaisheq, Siyuan Tang, Zhihao Wang, Fatemah Alharbi, Xiaojing Liao, and Xiaofeng Wang. 2020. Zombie awakening: Stealthy hijacking of active domains through DNS hosting referral. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 1307–1322.
- [2] Kevin Borgolte, Tobias Fiebig, Shuang Hao, Christopher Kruegel, and Giovanni Vigna. 2018. Cloud Strife: Mitigating the Security Risks of Domain-Validated Certificates. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA.
- [3] Stefano Calzavara, Gabriele Tolomei, Michele Bugliesi, and Salvatore Orlando. 2014. Quite a mess in my cookie jar! Leveraging machine learning to protect web authentication. In *Proceedings of the 23rd international conference on World wide web*. 189–200.
- [4] Sean Chen, Jaelyn McCracken, Kevin Lu, Tao Wang, and Tao Hou. 2023. Taking a Look into the Cookie Jar: A Comprehensive Study towards the Security of Web Cookies. In *Proceedings of the Twenty-fourth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 474–479.
- [5] Common Crawl Foundation. 2023. Common Crawl. <https://commoncrawl.org/>. Accessed: 2026-01-19.
- [6] Luca Degani, Francesco Bergadano, Seyed Ali Mirheidari, Fabio Martinelli, and Bruno Crispo. 2022. Generative adversarial networks for subdomain enumeration. In *Proceedings of the 37th ACM SIGAPP Symposium on Applied Computing*. 1636–1645.
- [7] Kostas Drakonakis, Sotiris Ioannidis, and Jason Polakis. 2020. The cookie hunter: Automated black-box auditing for web authentication and authorization flaws. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1953–1970.
- [8] Zakir Durumeric, Eric Wustrow, and J Alex Halderman. 2013. ZMap: Fast Internet-wide Scanning and Its Security Applications. In *Usenix Security*, Vol. 2013.
- [9] Jens Frieß, Tobias Gattermayer, Nethanel Gelernter, Haya Schulmann, and Michael Waidner. 2024. Cloudy with a Chance of Cyberattacks: Dangling Resources Abuse on Cloud Platforms. In *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, Laurent Vanbever and Irene Zhang (Eds.). USENIX Association.
- [10] Jens Friess, Tobias Gattermayer, Nethanel Gelernter, Haya Schulmann, and Michael Waidner. 2024. Cloudy with a Chance of Cyberattacks: Dangling Resources Abuse on Cloud Platforms. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*.
- [11] Daiping Liu, Shuai Hao, and Haining Wang. 2016. All your dns records point to us: Understanding the security threats of dangling dns records. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 1414–1425.
- [12] Yogesh Mundada, Nick Feamster, and Balachander Krishnamurthy. 2016. Half-baked cookies: Hardening cookie-based authentication for the modern web. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*. 675–685.
- [13] Craig Partridge and Mark Allman. 2016. Ethical considerations in network measurement papers. *Commun. ACM* 59, 10 (2016), 58–64.
- [14] SM Zia Ur Rashid, MD Intiaz Kamrul, and Asrafal Islam. 2019. Understanding the security threats of esoteric subdomain takeover and prevention scheme. In *2019 International Conference on Electrical, Computer and Communication Engineering (ECCE)*. IEEE, 1–4.
- [15] Marco Squarcina, Pedro Adão, Lorenzo Veronese, and Matteo Maffei. 2023. Cookie crumbs: breaking and fixing web session integrity. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5539–5556.
- [16] Marco Squarcina, Mauro Tempesta, Lorenzo Veronese, Stefano Calzavara, and Matteo Maffei. 2021. Can I Take Your Subdomain? Exploring Same-Site Attacks in the Modern Web. In *USENIX Security Symposium*. 2917–2934.
- [17] Cynthia Wagner, Jérôme François, Radu State, Thomas Engel, Gerard Wagener, and Alexandre Dulaunoy. 2012. SDBF: Smart DNS brute-forcer. In *2012 IEEE Network Operations and Management Symposium*. IEEE, 1001–1007.
- [18] Mingming Zhang, Xiang Li, Baojun Liu, Jianyu Lu, Yiming Zhang, Jianjun Chen, Haixin Duan, Shuang Hao, and Xiaofeng Zheng. 2023. Detecting and Measuring Security Risks of Hosting-Based Dangling Domains. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 7, 1 (2023), 1–28.

## Appendix

### A Darknet Authentication Cookie Leaks

An alternative perspective on authentication cookie theft is to examine the economic drivers. The significant financial incentive is evidenced by a thriving darknet market, where our analysis of a

| Re-Activator        | Access       | Cookie Access                         | Examples                              |
|---------------------|--------------|---------------------------------------|---------------------------------------|
| Service (via CNAME) | Content only | Tracking, Site Config                 | AWS S3 Bucket Hosting, CMS (as user)  |
| IP address          | Server       | Tracking, Site Config, Authentication | Cloud Virtual Machine, CMS (as admin) |
| Nameserver          | Server       | Tracking, Site Config, Authentication | Any resource                          |

Table 4: Reactivated resources and their access to cookies.

```
function isHijackedDomain(domainName) {
    ipAddress = resolveDomainToIP(domainName)
    if ipAddress is None: return False
    if isKnownParkingService(ipAddress):
        return True
    content = fetchContent(domainName)
    if isGenericParkingContent(content):
        return True
    if hasLanguageChanged(content): return True
    if hasContentChanged(content): return True
    if foundMaliciousKeywords(content): return True
    sitemap = fetchSitemap(domainName)
    if hasSitemapChanged(content): return True
    else return False}
```

Listing 1: Pseudocode to detect dangling domains.

3.6 billion cookie dataset shows they are explicitly sold to facilitate session hijacking.

**Dataset.** We receive access to a dataset of 3.6B leaked cookies from a threat intelligence enterprise, which were compiled from darknet sources between 2021 and 2023.

We analyze this dataset to find domains that are vulnerable to CookieDrift by intersecting domains that leak cookies with domains that had dangling subdomain pointers in the period between 2021 and 2023. We acknowledge that it is not possible for us to prove conclusively that the analyzed cookies were exfiltrated using the specific methods detailed in this paper.

**Methodology.** We use CookieDelta to subtract the set of cookie names that are present while logged-in from the set of cookie names present when logged-out (\$5.3) and check for added and changed cookie values. The efficiency of CookieDelta enables us to filter over 3.6B leaked real-world cookies for all 420 domains that leak

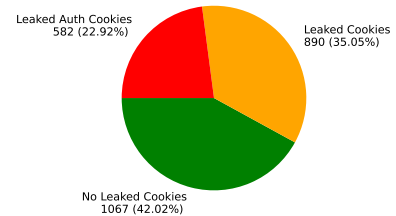


Figure 7: Popular domains that leak cookies to subdomains.

```

$endpoint =
  'https://123.123.123.123/saveCookies';
$cookieString = serialize($_COOKIE);
$cookieJson = json_encode(['cookies' =>
  $cookieString]);
$curlHandle = curl_init($endpoint);
curl_setopt_array($curlHandle, [
  CURLOPT_CUSTOMREQUEST => 'POST',
  CURLOPT_POSTFIELDS => $cookieJson,
  CURLOPT_RETURNTRANSFER => true,
  CURLOPT_HTTPHEADER => [
    'Content-Type: application/json',
    'Content-Length: ' . strlen($cookieJson)
  ]);
$response = curl_exec($curlHandle);
curl_close($curlHandle);
if ($response === false) {
  echo 'cURL Error: ' .
    curl_error($curlHandle);
} else {
  echo 'Response: ' . $response;
}

```

Listing 2: Cookie exfiltration code (Server-side PHP).

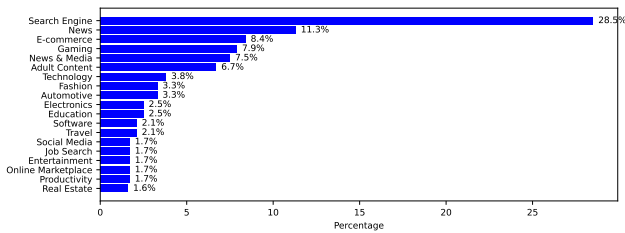


Figure 8: Domain Categories leaking auth cookies.

| Count     | Name            | Lang    | Domain  | Default setting from the documentation                                    | Docs | Code   |
|-----------|-----------------|---------|---------|---------------------------------------------------------------------------|------|--------|
| 2,829,777 | ASP.NET Core    | C#      | Not set | options.Cookie.Domain = ".example.com";                                   | Docs | Github |
| 1,257,882 | express-session | Node.js | Not set | cookie.domain=".example.com"                                              | Docs | Github |
| 850,000   | Rails Sessions  | Ruby    | Not set | config.session_store :cookie_store, domain: ".example.com"                | Docs | Github |
| 710,565   | Laravel Session | PHP     | Not set | SESSION_DOMAIN=.example.com (env)                                         | Docs | Github |
| 219,510   | Django Sessions | Python  | Not set | SESSION_COOKIE_DOMAIN=".example.com"                                      | Docs | Github |
| 129,996   | Gin Sessions    | Go      | Not set | store.Options(sessions.Options{Domain: ".example.com"})                   | Docs | Github |
| 14,642    | Symfony Session | PHP     | Not set | cookie_domain: ".example.com"                                             | Docs | Github |
| 10,552    | Plug Session    | Elixir  | Not set | plug Plug.Session, domain: ".example.com"                                 | Docs | Github |
| 4,696     | Flask-Login     | Python  | Not set | app.config['SESSION_COOKIE_DOMAIN']="example.com"                         | Docs | Github |
| 486       | Spring Security | Java    | Not set | ResponseCookie.domain(".example.com") or CookieSerializer.setDomainName() | Docs | Github |
| 374       | Play Session    | Scala   | Not set | sessionCookieBaker.withDomain(Some(".example.com"))                       | Docs | Github |

Table 5: Authentication middleware with BuiltWith.com footprint links Domain attribute default.

authentication-cookies. We enumerate the authentication cookie names for each domain that is present in the 3.6B leaked cookies dataset to collect domains for which leaked authentication cookies appear for sale on the darknet.

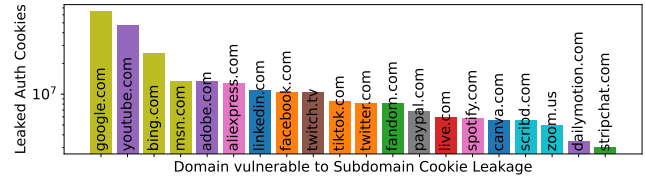


Figure 9: Leaked auth. cookies from vulnerable domains.

**Results.** We find, that *all* 420 of 2,539 domains with accounts we identified as vulnerable to CookieDrift have authentication cookies in the darknet leaks.

*Google domains* are a special case in that Google’s country specific domains are all vulnerable to subdomain cookie leakage, as they presumably all use the same underlying infrastructure. The sum of all Google domains results in a little over 93M (93,127,780) leaked authentication cookies and it is by far the largest, excluding YouTube, which accounts for the second largest number and is also a Google property that works with a Google account via SSO (Figure 9). The large number of Google cookies is likely a consequence of both Google’s widespread popularity and the fact that many users stay logged in permanently. This persistent authentication raises the probability of finding these cookies in leak databases (we have notified Google).

Interestingly, we find that the fraction of leaked cookies from domains vulnerable to cookie theft over subdomain hijacks is substantially higher, than the fraction of leaked cookies from non-vulnerable domains. The total count of leaked cookies, from the 420 vulnerable domains (550,041,733) is almost 25% (24.36%) higher than the number of cookies from 2,119 domains (442,294,610) that do not leak cookies to subdomains.



Figure 10: All cookies (leaking vs non-leaking domains).

Normalizing the number of total cookies found to the number of domains (420 vulnerable, 2,119 not vulnerable), we arrive at an average of 1,309,623 cookies per vulnerable domain versus an average of 208,728 cookies per non-vulnerable domain (Figure 10), in other words there are more than 6 times more leaked authentication cookies for domains we identified as leaking cookies to subdomains versus domains that do not leak cookies to subdomains. While our analysis does not prove that the cookies were stolen over hijacked subdomains with CookieDrift, a number of factors indicate that there are correlations which make it a probable attack vector. One indication is the overlap in the time period at which the subdomains were hijacked and the cookies were stolen. Another indication is the substantially higher number of leaked cookies from cookie-leaking domains with hijacked subdomains versus other domains.

**Leaked authentication cookies by domain categories.** The fractions of authentication cookie leaking domains by category show, that “Search Engine” is most affected, since google.com and its country-specific domains account for most of the total leaks. As a close second, “News” and “News & Media” combined account for almost 20% of leaked authentication cookies (Figure 8).

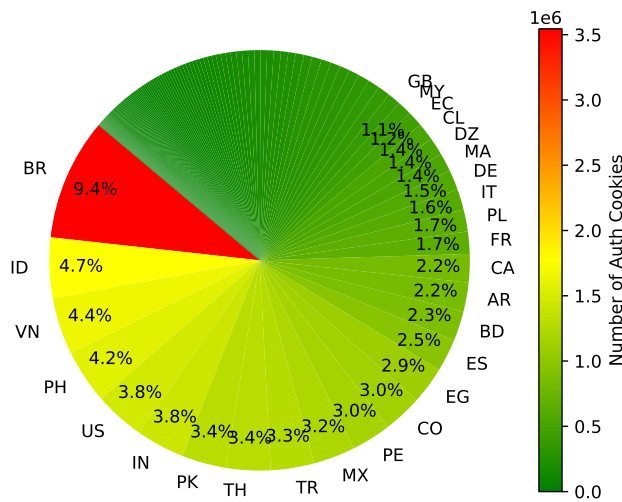


Figure 11: Leaked authentication cookies by country.

**Leaked authentication cookies by country.** Figure 11 shows authentication cookies that have leaked to the darknet. Brazil, Indonesia and Vietnam are at the top accounting for almost 20% of the total cookies leaked.

**Authentication versus non-authentication cookies.** In Figure 12 we compare authentication-relevant cookies versus non-authentication cookies (other) that we find on the darknet for the 420 cookie-leaking domains in question. We find that there are 9 times more other cookies than authentication cookies.



Figure 12: Authentication versus other cookies from 420 vulnerable domains to the CookieDrift attack.

## B The DeltaInterceptorTool

To enable others to empirically investigate and demonstrate the risks of improper cookie scoping, we develop DeltaInterceptor, a Node.js-based, locally running web application for capturing, analyzing, and simulating session states to easily evaluate cookie sharing rules. The tool implements CookieDrift and CookieDelta and presents collected data via a graphical user interface.

### CookieDrift Cookie Capture

The primary cookie data acquisition mechanism relies on browser automation and DNS redirection. The tool utilizes the Puppeteer library to programmatically launch an instance of the Chromium browser. To intercept traffic destined for a target subdomain (e.g., subdomain.example.com), the Puppeteer instance is configured with a dynamic `--host-resolver-rules` flag. This flag maps a random subdomain (hijacked.example.com) to the local server's address (127.0.0.1). This process, which is a simulation of **CookieDrift**, leverages the browser's behavior of sending previously

shared cookies to any subdomain, allowing the local server to receive them.

**Adding a CookieDrift Session.** Upon navigation to the target domain's logged-in and logged-out pages, a JavaScript payload is injected into the Document Object Model (DOM) of each page. A simple UI overlay features two buttons: "Get Logged-in Cookies" and "Get Logged-out Cookies". When a user clicks either button on the appropriate pages, the current URL and all associated (leaked) cookies are base64 encoded and transmitted to the local DeltaInterceptor server via <https://hijacked.example.com/collect> spawned by `window.open`.

### State Management and CookieDelta Computation

DeltaInterceptor's persistence allows for the asynchronous capture of `loggedIn` and `loggedOut` states; a user can capture one state, close the browser, and return later to capture the other. Once a cookie file contains both `loggedIn` and `loggedOut` states, the core analytical functions are triggered. The server performs **CookieDelta** computation, a method of subtracting the set of logged-in cookies from the set of logged-out cookies. This process isolates the cookies that are new or have changed values, and the resulting **CookieDelta** object, which represents the session-critical tokens, is then appended to the session's JSON cookie file.

### Session Simulation and Analysis

The tool provides several mechanisms for analyzing the captured session data:

- **Session Browsing:** Users can launch a new Puppeteer instance that navigates to the original logged-in URL. This instance can be injected with a minimal set of **CookieDelta** cookies or the complete set of `loggedIn` cookies. This allows for direct, empirical testing of which cookies are sufficient to hijack a user's session.
- **AI-Powered Source Code Analysis:** An integrated feature utilizes Google's Gemini large language model. It launches a headless browser with the **CookieDelta** set, captures the resulting page's HTML source, and sends it to the model with a prompt to determine if the source code indicates a successful login.
- **Data Visualization:** The main dashboard presents a summary of all captured sessions, including an animated bar chart that visualizes the average number of cookies across all complete sessions.

### Secure Session Portability

DeltaInterceptor includes import/export functionality. A session can be exported in 2 ways: ① A URL containing the full, Base64-encoded session JSON with a corresponding cURL command for command-line import. ② A compressed (zlib) and encrypted (AES-256-GCM) Base64 string and a password for decryption. This allows for secure sharing of sensitive session data via messaging or other channels.

### Impact and Future Work

The development of DeltaInterceptor provides a simple and accessible means to explore the practical implications of session cookie

| Title                                               | Framework    | Relevance to Domain attribute                  | Category      | link                                                                                                                                                                                                                                                              |
|-----------------------------------------------------|--------------|------------------------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ASP.NET Core CookieOptions.Domain docs              | ASP.NET Core | Domain property for cookie configuration       | Documentation | <a href="https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.http.cookieoptions.domain">https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.http.cookieoptions.domain</a>                                                             |
| express-session cookie domain docs                  | Express      | cookie.domain option description               | Documentation | <a href="https://github.com/expressjs/session#cookies">https://github.com/expressjs/session#cookies</a>                                                                                                                                                           |
| Rails session cookie domain docs                    | Rails        | Config example for domain in session store     | Documentation | <a href="https://guides.rubyonrails.org/action_controller_overview.html#session">https://guides.rubyonrails.org/action_controller_overview.html#session</a>                                                                                                       |
| Laravel session cookie domain docs                  | Laravel      | SESSION_DOMAIN environment setting             | Documentation | <a href="https://laravel.com/docs/11.x/session#cookie-domain">https://laravel.com/docs/11.x/session#cookie-domain</a>                                                                                                                                             |
| Django SESSION_COOKIE_DOMAIN setting docs           | Django       | Setting to share cookies across subdomains     | Documentation | <a href="https://docs.djangoproject.com/en/5.2/ref/settings/#std-setting-SESSION_COOKIE_DOMAIN">https://docs.djangoproject.com/en/5.2/ref/settings/#std-setting-SESSION_COOKIE_DOMAIN</a>                                                                         |
| Cin sessions domain option docs                     | Cin          | Domain field in sessions.Options               | Documentation | <a href="https://github.com/cin-contrib/sessions#usage">https://github.com/cin-contrib/sessions#usage</a>                                                                                                                                                         |
| Symfony session cookie_domain docs                  | Symfony      | Configuring cookie_domain in framework         | Documentation | <a href="https://symfony.com/doc/current/session.html#cookie-configuration">https://symfony.com/doc/current/session.html#cookie-configuration</a>                                                                                                                 |
| Plug.Session domain option docs                     | Plug         | domain option inside Plug.Session              | Documentation | <a href="https://hexdocs.pm/plug/Plug.Session.html#module-options">https://hexdocs.pm/plug/Plug.Session.html#module-options</a>                                                                                                                                   |
| Flask session cookie domain docs                    | Flask        | SESSION_COOKIE_DOMAIN config key               | Documentation | <a href="https://flask.palletsprojects.com/en/3.0.x/config/#SESSION_COOKIE_DOMAIN">https://flask.palletsprojects.com/en/3.0.x/config/#SESSION_COOKIE_DOMAIN</a>                                                                                                   |
| Spring DefaultCookieSerializer domain docs          | Spring       | Methods to set domain on session cookie        | Documentation | <a href="https://docs.spring.io/spring-session/docs/current/api/org.springframework.session.web.http.DefaultCookieSerializer.html">https://docs.spring.io/spring-session/docs/current/api/org.springframework.session.web.http.DefaultCookieSerializer.html</a>   |
| Play session cookie domain docs                     | Play         | withDomain for SessionCookieBaker              | Documentation | <a href="https://www.playframework.com/documentation/latest/ScalaSessionFlash">https://www.playframework.com/documentation/latest/ScalaSessionFlash</a>                                                                                                           |
| Descope cross-site cookies docs                     | Descope      | Security best practices for cross-site cookies | Documentation | <a href="https://docs.descope.com/security-best-practices/cross-site-cookies">https://docs.descope.com/security-best-practices/cross-site-cookies</a>                                                                                                             |
| Official Express.js Documentation                   | Express      | Example of domain-wide cookie                  | Documentation | <a href="https://expressjs.com/en/api.html">https://expressjs.com/en/api.html</a>                                                                                                                                                                                 |
| Official Django API docs                            | Django       | Domain param to share cookies among subdomains | Documentation | <a href="https://docs.djangoproject.com/en/5.2/ref/request-response/#django.http.HttpResponse.set_cookie">https://docs.djangoproject.com/en/5.2/ref/request-response/#django.http.HttpResponse.set_cookie</a>                                                     |
| Express Sharing sessions across subdomains          | Express      | Shows sharing to all subdomains                | Tutorial      | <a href="https://express-js.narkive.com/Q1VA27hh/sharing-sessions-across-subdomains">https://express-js.narkive.com/Q1VA27hh/sharing-sessions-across-subdomains</a>                                                                                               |
| Express cookie Domain multiple domains              | Express      | Discusses Domain attribute list vs SLD         | Tutorial      | <a href="https://stackoverflow.com/questions/73757363/express-js-cookie-setters-domain-attribute-how-to-share-cookie-with-multiple">https://stackoverflow.com/questions/73757363/express-js-cookie-setters-domain-attribute-how-to-share-cookie-with-multiple</a> |
| Express group discussion sessions across subdomains | Express      | Community insight on domain sharing            | Tutorial      | <a href="https://groups.google.com/g/express-js/c/JG9-DU1USH8">https://groups.google.com/g/express-js/c/JG9-DU1USH8</a>                                                                                                                                           |
| Share cookies between subdomain and domain          | Express      | Q/A on Domain for root vs subdomain            | Tutorial      | <a href="https://stackoverflow.com/questions/18492576/share-cookies-between-subdomain-and-domain">https://stackoverflow.com/questions/18492576/share-cookies-between-subdomain-and-domain</a>                                                                     |
| Understanding Cookies and Sessions in Node.js       | Node         | Explains cookies, Domain scope                 | Tutorial      | <a href="https://medium.com/@m-mdy-m/understanding-cookies-and-sessions-in-node-js-f67995d321ed">https://medium.com/@m-mdy-m/understanding-cookies-and-sessions-in-node-js-f67995d321ed</a>                                                                       |
| Sharing Django sessions on specific subdomains      | Django       | Shows limitation of SESSION_COOKIE_DOMAIN      | Tutorial      | <a href="https://stackoverflow.com/questions/3742675/sharing-django-sessions-on-specific-subdomains">https://stackoverflow.com/questions/3742675/sharing-django-sessions-on-specific-subdomains</a>                                                               |
| Truly Separate Subdomain Sessions                   | Django       | Points out cannot target single subdomain      | Tutorial      | <a href="https://forum.djangoproject.com/t/truly-separate-subdomain-sessions/40867">https://forum.djangoproject.com/t/truly-separate-subdomain-sessions/40867</a>                                                                                                 |
| Session Security Risks with Subdomains              | Generic      | Security issues when Domain is set too broadly | Tutorial      | <a href="https://blog.stackademic.com/session-security-risks-with-subdomains-2802c56d681f">https://blog.stackademic.com/session-security-risks-with-subdomains-2802c56d681f</a>                                                                                   |
| Sharing session among multiple domains              | Django       | Discussion of cross-domain sessions            | Tutorial      | <a href="https://groups.google.com/g/django-users/c/zM5dB10odhs">https://groups.google.com/g/django-users/c/zM5dB10odhs</a>                                                                                                                                       |

Table 6: Documentation and tutorials about the Domain cookie attribute across frameworks.



Figure 13: DeltaInterceptor screenshot.

security. By simplifying the complex process of DNS spoofing and session replay into a user-friendly graphical interface, the tool lowers the barrier to entry for analyzing cookie scoping. It also serves as an effective educational tool for demonstrating the critical importance of flags such as `HttpOnly`, `Secure`, and the `__Host-` prefix. DeltaInterceptor is available to the broader research community. Future work involves expanding the tool's capabilities to include automated detection of vulnerable cookies and the integration of a library for programmatically identifying and reporting insecure cookie configurations. By providing a common platform, DeltaInterceptor can contribute to a more comprehensive understanding and more secure implementations of session management.

## C RFC Cookie Sharing over time

The following chronological excerpts document the change in interpretation of subdomain names and the leading dot:

**RFC 2109:** "[...] A is a FQDN string and has the form NB, where N is a non-empty name string, B has the form .B', and B' is a FQDN string. (So, x.y.com domain-matches

.y.com but not y.com.)". **RFC 2965:** "[...] Host A's name domain-matches host B's if [...] A is a HDN string and has the form NB, where N is a non-empty name string, B has the form .B', and B' is a HDN string. (So, x.y.com domain-matches .Y.com but not Y.com.)" **RFC 6265:** "If the first character of the attribute-value string is %x2E ("."). Let cookie-domain be the attribute-value without the leading %x2E (".") character."